



Study Programme 42484 — Information Systems (BSc)
Department of Natural Sciences and Computer Engineering
RIGA NORDIC UNIVERSITY (RNU)

Integrating Artificial Intelligence Models for MEDLINE Clinic Management

BACHELOR'S THESIS

Student: Turgunov Adhamjon

Supervisor: Dr. Jelena Caiko, PhD

Riga 2026

Abstract

This thesis describes MEDLINE, a clinic management system built using Flutter and Firebase. The system addresses a concrete operational problem: small and medium-sized private clinics lack affordable, unified tools for patient registration, real-time multi-department monitoring, and structured automated reporting. Paper-based and fragmented digital workflows cause data loss, delayed clinical decisions, and unauditable financial records.

The thesis consists of four chapters. Chapter 1 reviews the theoretical background covering healthcare information systems, cross-platform mobile development with Flutter, role-based access control (RBAC), Cloud Firestore’s NoSQL document model, and knowledge-based clinical decision support. Chapter 2 analyses the problem domain, identifies six stakeholder groups, and elicits ten functional and six non-functional requirements. Chapter 3 describes the system design and implementation: the Firebase backend, Flutter widget architecture, Provider state management, six role-specific dashboards, a rule-based symptom-to-disease diagnostic decision-support engine, multilingual UI, and client-side PDF report generation. Chapter 4 evaluates the implemented system against all stated requirements, discusses architectural trade-offs and limitations, and proposes six directions for future development.

The methodological basis of the study includes the analysis of scientific literature, structured requirements elicitation, iterative software construction, and end-to-end functional verification across three target platforms. The research hypothesis — that a single Flutter/Firebase codebase can fully provide patient registration, departmental monitoring, and automated reporting across Android, iOS, and the Web — was confirmed: all ten functional requirements were verified as satisfied on Android, the Web, and Windows builds compiled from one shared codebase, with dashboard load times measured below the three-second target.

The Bachelor Paper is written in 40 pages and contains 11 tables, 8 figures, 2 algorithms, 5 listings, 2 annexes, and a list of references with 26 sources.

Keywords: clinic management system, Flutter, Cloud Firestore, role-based access control, cross-platform mobile development, clinical decision support, electronic health records.

Key words

The following key concepts characterise the topic and content of this thesis:

- **Clinic management system** — an integrated application that consolidates patient registration, departmental operations, and reporting.
- **Flutter** — a cross-platform UI toolkit that compiles a single Dart codebase to Android, iOS, and Web targets.
- **Cloud Firestore** — a document-oriented NoSQL database with real-time synchronisation, used as the system backend.
- **Role-based access control (RBAC)** — a security model assigning permissions to roles rather than to individual users.
- **Cross-platform mobile development** — writing application logic once and deploying it to multiple platforms without code duplication.
- **Clinical decision support** — software that links patient data to a medical knowledge base to assist clinical reasoning.
- **Electronic health records (EHR)** — the digital clinical data layer maintained throughout the patient journey.

Contents

Abstract	1
Key words	2
Introduction	8
1 Literature Review and Theoretical Background	10
1.1 Healthcare Information Systems	10
1.2 Cross-Platform Mobile Development	11
1.2.1 Flutter Framework	11
1.2.2 Dart Programming Language	11
1.3 Role-Based Access Control (RBAC)	12
1.4 Cloud Firestore and NoSQL Architecture	12
1.5 State Management with Provider	13
1.6 Clinical Decision Support and AI in Healthcare	13
1.7 Automated PDF Report Generation	14
1.8 Justification of Technology Choices	14
1.9 Chapter Summary	15
2 Problem Domain Analysis and System Requirements	16
2.1 Target Environment Characteristics	16
2.2 Operational Pain Points	16
2.3 Stakeholder Analysis	17
2.4 Functional Requirements	17
2.5 Non-Functional Requirements	19
2.6 Comparison with Existing Systems	19
2.7 Sub-Conclusions	19
3 System Design and Implementation	21
3.1 System Architecture Overview	21
3.2 Firebase Backend Design	22
3.2.1 Firestore Data Model	22

3.2.2	Authentication and Role Routing	22
3.3	Flutter Application Structure	25
3.4	Role-Specific Dashboards	25
3.4.1	Admin Dashboard	25
3.4.2	Doctor Dashboard	27
3.4.3	Receptionist Dashboard	27
3.4.4	Laboratory Dashboard	28
3.4.5	Pharmacy Dashboard	28
3.4.6	Hospitalization Dashboard	28
3.5	Diagnostic Decision-Support Module	28
3.5.1	Knowledge Base	29
3.5.2	Symptom-Matching and Scoring	29
3.6	Multilingual Support	32
3.7	Theme System	32
3.8	Automated PDF Report Generation	32
3.9	Sub-Conclusions	32
4	System Evaluation, Discussion, and Future Work	33
4.1	Functional Requirement Evaluation	33
4.2	Non-Functional Requirement Evaluation	34
4.2.1	Performance (NFR-01)	34
4.2.2	Security (NFR-03)	34
4.2.3	Portability (NFR-05)	34
4.2.4	Usability (NFR-04)	35
4.3	Discussion of Architectural Trade-offs	35
4.4	Hypothesis Validation	35
4.5	Limitations of the Study	35
4.6	Future Work	36
4.7	Sub-Conclusions	36
	Conclusions	37
	References	39
A	Key Source Code Listings	41
A.1	AuthWrapper — Role-Based Routing	41
A.2	LanguageProvider — Runtime Translation	41
A.3	AppDateUtils — Locale-Aware Timestamp Formatting	42
B	Firestore Data Schema	43

Affirmation	45
Acknowledgments	46

List of Figures

1.1	The layered Flutter architecture exploited by MEDLINE: one application codebase runs unchanged on every embedder.	12
2.1	Stakeholder–role overview: six actors interact with MEDLINE, each through a dedicated role dashboard.	18
3.1	Five-layer architecture of the MEDLINE clinic-management system.	22
3.2	Document-oriented Firestore data model with seven top-level collections.	23
3.3	MEDLINE login screen: the email and password authentication entry point with a language selector, rendered from the shared Flutter codebase.	26
3.4	Admin dashboard: clinic-wide statistics showing total patients, total revenue, paid and unpaid counts, and per-doctor breakdowns with Today / Week / Month filters.	26
3.5	Doctor dashboard: the daily waiting-patient queue showing each patient’s complaint, address, payment status, and the Add Diagnosis action.	27
3.6	Receptionist dashboard: the patient-registration form with quick-access modules (DMED.UZ, diagnostic agent, and hospitalization).	27
3.7	Hospitalization dashboard: the ward room list showing each room’s availability status, bed count, type, and daily price, with an Admit Patient action.	28
3.8	Diagnostic decision-support pipeline: from symptom selection to ranked output.	31
4.1	Measured dashboard cold-start load times against the 3-second NFR-01 target.	34

List of Tables

1.1	MEDLINE user roles and primary responsibilities.	13
2.1	Operational pain points in manual and fragmented clinic workflows.	17
2.2	Functional requirements of the MEDLINE system.	18
2.3	Non-functional requirements of the MEDLINE system.	19
2.4	Comparison of MEDLINE with selected clinic-management approaches. . .	20
3.1	Firestore collections and key document fields.	24
3.2	Role dashboards and primary Firestore data sources.	26
4.1	Functional requirement evaluation results.	33
B.1	<code>users</code> collection schema.	43
B.2	<code>patients</code> collection schema.	43
B.3	<code>lab_requests</code> collection schema.	44

Introduction

Topicality of the problem. Healthcare facilities worldwide face increasing pressure to improve service quality, reduce administrative errors, and comply with digital health standards. According to the World Health Organization, digital interventions in health systems can significantly improve patient safety, resource utilisation, and clinical outcomes [1]. Despite this, many small and medium-sized private clinics in Central Asia and Eastern Europe continue to rely on paper-based registration, spreadsheet logs, and disconnected departmental software. This fragmentation leads to data inconsistencies, delayed diagnoses, and uncontrolled financial reporting. MEDLINE was developed as a direct response: a unified, cross-platform clinic management application consolidating patient registration, departmental monitoring, pharmacy and laboratory tracking, a symptom-based diagnostic aid, and automated PDF reporting into a single system deployable on Android, iOS, and the Web.

Aim of the study. The aim is to design and implement an integrated registration and monitoring platform for MEDLINE clinics that provides complete oversight of clinical and accounting data, including automated daily, monthly, and yearly reports.

Object of the study. Patient registration, and the storage and monitoring of collected clinical and financial data, in the context of digitalising a clinic's internal processes.

Subject of the study. The design and development of an integrated patient-registration, diagnostic, and real-time monitoring system for the digitalisation of clinic operations.

Research problem. Many small clinics do not maintain a unified patient database. Numerous facilities still use traditional paper registers to record patient visits, which consumes significant staff time and physical storage and prevents real-time cross-departmental visibility and reliable financial auditing.

Tasks of the study. The work is organised around four tasks that correspond directly to the four chapters of the thesis:

1. Review theoretical approaches and relevant literature on healthcare information systems, cross-platform mobile development, role-based access control, NoSQL data modelling, and clinical decision support.
2. Analyse the problem domain of small private clinics, identify the stakeholder groups, and elicit the functional and non-functional requirements of the system.
3. Design and implement MEDLINE — automated patient registration, role-based departmental modules, a symptom-to-disease diagnostic decision-support algorithm, real-time monitoring, and automated PDF reporting — from a single Flutter/Firebase codebase.
4. Evaluate the implemented system against the stated requirements, validate the research hypothesis, and propose directions for improvement.

Hypothesis. If a clinic management system is developed using Flutter and Firebase, then a single shared codebase can fully provide patient registration, departmental monitoring, and automated reporting functions across Android, iOS, and Web platforms.

Research methods. The study applies: (1) *data collection* — systematic gathering of information about patients and clinic processes; (2) *database design* — construction of an electronic store for patient records, diagnoses, and payment information; (3) *algorithm development* — automation of symptom-based disease ranking and specialist recommendation; and (4) *testing and verification* — checking system functionality, identifying defects, and applying corrections. These are complemented by scientific-literature analysis, comparative analysis of existing products, and iterative software construction.

Structure of the thesis. The thesis consists of four chapters corresponding to the four tasks. Chapter 1 provides the theoretical foundation (Task 1). Chapter 2 analyses the problem domain and system requirements (Task 2). Chapter 3 describes the design and implementation of MEDLINE (Task 3). Chapter 4 evaluates the system and proposes future improvements (Task 4). The thesis concludes with a synthesised summary, a reference list, and two appendices containing supplementary source-code listings and the Firestore data schema.

Approbation of the study. The MEDLINE system and the design decisions reported here were prepared as the practical component of the bachelor programme and reviewed by the thesis supervisor at the Department of Natural Sciences and Computer Engineering.

Chapter 1

Literature Review and Theoretical Background

This chapter establishes the theoretical foundations on which MEDLINE is built. It defines the core concepts the thesis relies on, contrasts the principal technical approaches, and justifies the technology choices that shape the design in Chapter 3. Each foundational term is introduced once and used consistently throughout the remainder of the thesis.

1.1 Healthcare Information Systems

Healthcare information systems (HIS) are integrated systems designed to manage administrative, financial, and clinical data within healthcare organisations [2]. Wager, Lee, and Glaser categorise HIS into three layers: clinical information systems (CIS), administrative information systems (AIS), and management information systems (MIS) [2]. The adoption of HIS reduces medical errors, accelerates patient throughput, and improves the auditability of clinical decisions [3].

Electronic health records (EHR) represent the clinical data layer of a HIS. Kruse et al. conducted a systematic review demonstrating that EHR adoption supports population health management through longitudinal patient-data aggregation and real-time alerting [3]. The WHO recommends digital health interventions as a cornerstone strategy for strengthening primary healthcare in low- and middle-income countries [1]. Mesko et al. argue that the transformation towards digital health is not merely technological but cultural, requiring organisational change alongside software deployment [4].

MEDLINE targets the intersection of AIS and CIS functions: it manages patient registration (AIS) while surfacing real-time departmental status to clinical staff (CIS), making it a lightweight, integrated HIS tailored for small private clinics that cannot afford full-scale enterprise systems.

1.2 Cross-Platform Mobile Development

Cross-platform frameworks allow developers to write application logic once and compile to multiple target platforms, reducing development time and maintenance overhead [5]. Three principal paradigms exist: web-based hybrid approaches (Ionic, Cordova), JavaScript-bridge architectures (React Native), and compiled native-UI frameworks such as Flutter [6].

Nawrocki et al. conducted a performance and developer-experience comparison of native, React Native, and Flutter applications, concluding that Flutter most closely matches native rendering performance owing to its compiled Dart code and proprietary Skia/Impeller graphics engine [5]. Biorn-Hansen et al. examined progressive web apps as a unifying strategy but found offline capability and hardware-access limitations [7]. Majchrzak et al. concluded that framework selection must be driven by project requirements rather than by a single universally superior choice [8].

1.2.1 Flutter Framework

Flutter is an open-source UI toolkit developed by Google that compiles Dart source code to native ARM or JavaScript targets [9]. Its architectural model separates three layers, shown in Figure 1.1: the Dart framework (widgets, rendering, animation), the Flutter engine (C++, Skia/Impeller), and platform-specific embedders [10]. This design isolates business logic from platform details — a property advantageous for healthcare applications requiring multi-platform deployment.

Flutter’s “everything is a widget” philosophy means every UI element is a widget composed declaratively into a tree the framework reconciles efficiently on each frame [11]. MEDLINE exploits this cross-platform compilation to deliver identical business logic and UI components across Android, iOS, and the Web from a single `lib/` directory, as specified in the `pubspec.yaml` project manifest, following established practice for production Flutter applications [22].

1.2.2 Dart Programming Language

Dart is a strongly typed, ahead-of-time compiled language designed for client-side development [12]. Its `async/await` syntax maps naturally to Firebase’s asynchronous stream and future APIs. In MEDLINE, `AuthWrapper` uses a `StreamBuilder` to listen to Firebase Authentication state changes and a `FutureBuilder` to resolve the Firestore user-role document — both idiomatic Dart concurrency patterns that keep the UI responsive during network operations.

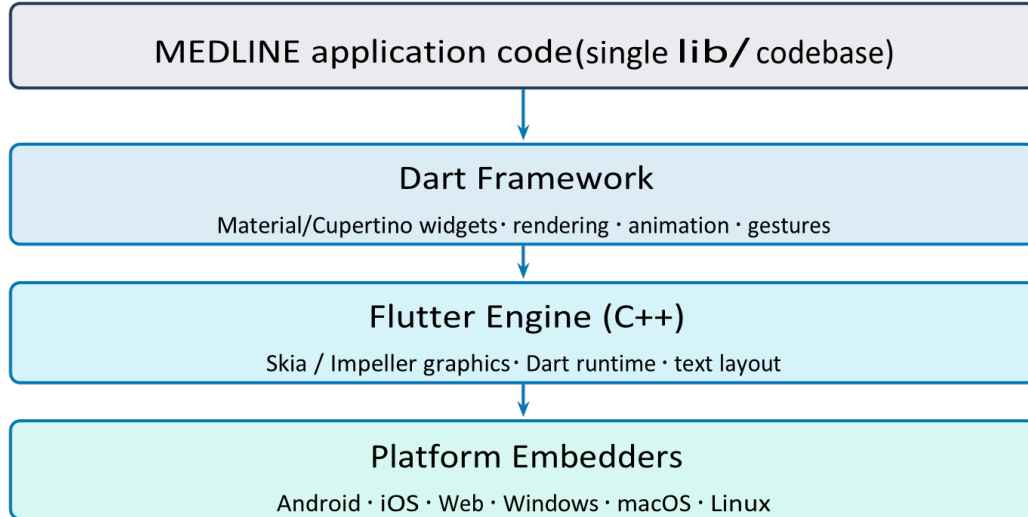


Figure 1.1. The layered Flutter architecture exploited by MEDLINE: one application codebase runs unchanged on every embedder.

Source: author-created illustration based on the Flutter architectural overview [10].

1.3 Role-Based Access Control (RBAC)

Role-based access control is a security model in which permissions are assigned to roles rather than directly to individual users [13]. Sandhu et al. formalised the RBAC96 model with four components: users, roles, permissions, and sessions. Users are assigned to roles; roles are granted permissions; sessions activate a subset of a user’s assigned roles at runtime [13].

Fernandez-Aleman et al. reviewed security mechanisms in electronic health records and identified RBAC as the dominant access-control strategy in clinical software, noting that improper access control is the leading cause of EHR data breaches [14]. MEDLINE implements a simplified RBAC96 subset: six named roles (Table 1.1) are stored per user in Firestore. The **AuthWrapper** reads the authenticated user’s role document and routes to the corresponding dashboard, enforcing role isolation at the UI-routing layer.

1.4 Cloud Firestore and NoSQL Architecture

NoSQL databases abandon the relational table model in favour of flexible document, key-value, column-family, or graph structures [15]. Cloud Firestore is a document-oriented NoSQL database offered as part of Google Firebase [16]. Data is organised into collections of documents — JSON-like key-value maps that can contain nested sub-collections. Firestore’s real-time snapshot listeners push incremental updates to connected clients without polling, a property exploited in all MEDLINE dashboard widgets via **StreamBuilder**.

Key Firestore characteristics relevant to MEDLINE are: offline persistence (a local cache survives connectivity loss, critical in clinics with unstable internet), security rules

Table 1.1. MEDLINE user roles and primary responsibilities.

Role	Primary responsibilities
Admin	Monitor clinic-wide statistics, manage user accounts and role assignment, export administrative PDF reports.
Receptionist	Search and register patients, schedule appointments, record payment method.
Doctor	Review the daily patient queue, write consultation notes, assign diagnosis codes, issue referrals.
Laboratory	Process diagnostic referrals, enter and complete test results, generate result sheets.
Pharmacy	View prescriptions, confirm dispensing, flag out-of-stock items.
Hospitalization	Assign beds, monitor inpatient daily status, record discharge.

Source: author, derived from the implemented role routing in `auth_wrapper.dart`.

(server-side enforcement of read/write permissions by authenticated UID), and composite-index support for multi-field queries such as filtering patients by registration date and department [16].

1.5 State Management with Provider

State management is a central architectural concern in Flutter: the framework rebuilds widget subtrees on state change, so uncontrolled propagation causes redundant rebuilds and tangled data flows [11]. The Provider package wraps Flutter’s `InheritedWidget` into a dependency-injection container that notifies registered listeners on state mutation [17].

MEDLINE uses two global providers. `ThemeProvider` manages light/dark-mode preferences persisted via `SharedPreferences`, exposing derived colour properties (`cardBackground`, `inputFill`, `textPrimary`) so that screen widgets remain palette-agnostic. `LanguageProvider` holds the active locale (UZB, ENG, RUS, or KYR) with a translation map covering more than two hundred UI string keys. Both providers are injected at the root via `MultiProvider` in `main.dart`, making them accessible anywhere in the widget tree without manual prop-drilling.

1.6 Clinical Decision Support and AI in Healthcare

A clinical decision support system (CDSS) is software that links patient-specific information to a structured medical knowledge base in order to generate case-specific assessments or recommendations that assist clinical reasoning [26]. Two broad CDSS paradigms are commonly distinguished. *Knowledge-based* systems apply curated rules and explicit symptom–disease associations (the classical expert-system branch of artificial intelligence); *non-knowledge-based* systems instead learn statistical patterns from

data using machine-learning models [26]. Knowledge-based systems remain widely used in resource-limited settings because they are transparent, require no training data, and are inexpensive to deploy, whereas machine-learning systems demand large labelled datasets and careful validation before clinical use.

MEDLINE integrates a lightweight knowledge-based diagnostic decision-support model. A curated knowledge base of thirty-one diseases — each associated with a characteristic symptom set and a recommended specialist — drives a scoring function that ranks candidate diagnoses for a selected combination of symptoms (detailed in Section 3.5). This positions the engine as an AI-assisted *triage aid* that suggests likely conditions and the appropriate specialist, rather than as an autonomous diagnostic authority; a migration toward data-driven machine-learning models is identified as future work (Section 4.6). The choice of a rule-based model is consistent with the constraints of the target clinics, which lack the labelled clinical datasets and computational infrastructure required to train and serve machine-learning classifiers.

1.7 Automated PDF Report Generation

Automated document generation is a standard requirement in clinical settings — for discharge summaries, prescription records, and administrative reports. The `pdf` Dart package allows programmatic construction of PDF pages using a box model similar to Flutter’s widget system [18]. MEDLINE uses this package together with `printing` to render and share reports directly from the application without server-side processing, keeping all patient data on-device during rendering — a privacy advantage in healthcare contexts.

1.8 Justification of Technology Choices

The selection of Flutter over React Native is justified by its superior rendering performance in benchmark comparisons [5], a single codebase for Android, iOS, and Web deployment, and strong support for custom UI components — important for the role-specific dashboard designs in MEDLINE. Firebase was chosen over a self-hosted backend because the target clinics lack dedicated IT infrastructure; Firestore’s real-time listeners replace polling and reduce complexity [21]. The Provider package was chosen over Bloc or Riverpod because of its minimal boilerplate and sufficient capability for the two global state concerns (theme and language) that MEDLINE requires [17]. Client-side PDF generation was preferred over server-side rendering to eliminate a backend dependency and keep the system fully serverless. Finally, a knowledge-based diagnostic model was preferred over a machine-learning classifier because it is transparent and deployable without training data [26].

1.9 Chapter Summary

This chapter established the theoretical foundations underpinning MEDLINE. Healthcare information systems provide the clinical and administrative context; Flutter justifies the single-codebase cross-platform strategy; RBAC96 grounds the six-role security model; Firestore’s real-time document model supports the live dashboard architecture; the Provider pattern organises global application state; and knowledge-based clinical decision support frames the symptom-matching diagnostic engine. Each technology choice was explicitly justified against the project constraints. These foundations motivate the requirements analysis in Chapter 2 and the design decisions described in Chapter 3.

Chapter 2

Problem Domain Analysis and System Requirements

This chapter analyses the operational environment that MEDLINE targets, catalogues the deficiencies of current clinic workflows, identifies the stakeholders, and derives the functional and non-functional requirements that drive the design in Chapter 3.

2.1 Target Environment Characteristics

MEDLINE targets small and medium-sized private clinics operating multiple service departments under a single administrative umbrella. A typical such facility processes between fifty and three hundred patient visits per day across departments including general consultation, laboratory diagnostics, in-patient care, and pharmacy. The defining operational characteristic is the need for cross-departmental data visibility: a receptionist registers a patient; a doctor creates a referral; a laboratory technician uploads results; a pharmacist dispenses medication — all events must be traceable to a single patient record in real time.

Prior to MEDLINE, clinics in this category typically employed one or more of: paper ledgers and handwritten patient cards; general-purpose spreadsheet software maintained independently per department; or disconnected desktop billing software without clinical-data integration. Each approach introduces distinct failure modes analysed in Section 2.2.

2.2 Operational Pain Points

Table 2.1 catalogues the principal problems observed in manual and semi-digital clinic workflows that directly informed the MEDLINE requirements.

Table 2.1. Operational pain points in manual and fragmented clinic workflows.

Pain point	Consequence
Paper ledgers and handwritten cards	Illegible or lost records; no search; large physical storage requirement.
Independent per-department spreadsheets	No single source of truth; cross-department data inconsistency and duplication.
Disconnected billing software	Financial records cannot be reconciled with clinical activity; weak auditability.
Manual report compilation	Daily, monthly, and yearly reports are slow to produce and error-prone.
No real-time patient queue	Doctors are unaware of newly registered and waiting patients; consultation delays.
No automated backup or access control	Risk of data loss and uncontrolled access to sensitive patient information.

Source: author, based on analysis of the target clinic environment.

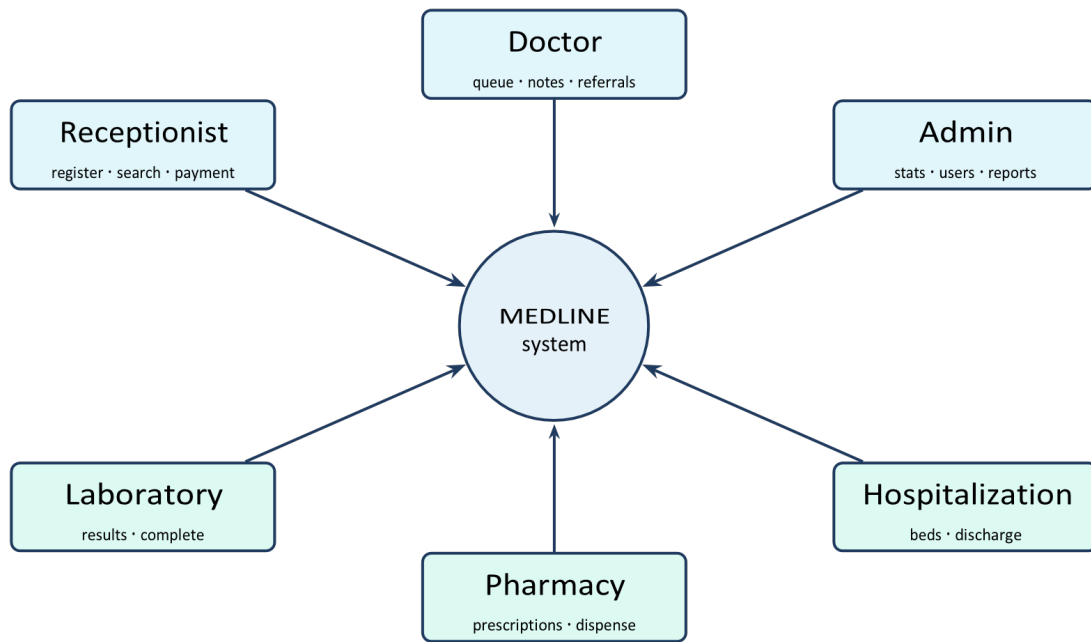
2.3 Stakeholder Analysis

Six stakeholder groups were identified, each corresponding to one MEDLINE role. Each group interacts with the system through a dedicated dashboard exposing only the data and actions relevant to that role, as summarised in Figure 2.1.

The *receptionist* is the entry point of every patient journey, with the primary use cases of searching existing patients, registering new patients with demographic data, scheduling appointments, and recording the payment method. The *doctor* consults registered patients: viewing the assigned day queue, writing consultation notes with ICD-10 diagnosis codes, issuing referrals to laboratory or pharmacy, and reviewing visit history. The *laboratory* technician processes diagnostic referrals: viewing pending test requests, entering quantitative or qualitative results, marking results as ready, and generating PDF result sheets. The *pharmacy* staff manage medication dispensing: viewing incoming prescriptions, confirming dispensing, recording stock deductions, and flagging out-of-stock items. The *hospitalization* department manages inpatient care: assigning patients to beds, monitoring daily status, and recording discharge. Finally, the *admin* monitors the entire clinic: viewing real-time aggregate statistics, managing user accounts and role assignments, and exporting administrative PDF reports.

2.4 Functional Requirements

Based on the stakeholder analysis and the observed workflow problems, the functional requirements in Table 2.2 were defined for MEDLINE following standard requirements-engineering practice [25].



Source: author-created use-case overview.

Figure 2.1. Stakeholder–role overview: six actors interact with MEDLINE, each through a dedicated role dashboard.

Table 2.2. Functional requirements of the MEDLINE system.

ID	Requirement
FR-01	Authenticate users via email and password and restrict access to role-specific views.
FR-02	Allow a receptionist to register a new patient with name, date of birth, gender, contact details, and visit purpose.
FR-03	Display a real-time patient queue on the doctor dashboard, updated without manual page refresh.
FR-04	Allow doctors to record consultation notes, assign an ICD-10 diagnosis code, and issue referrals to laboratory or pharmacy.
FR-05	Allow laboratory staff to enter test results and mark requests as completed.
FR-06	Allow pharmacy staff to view prescriptions and confirm dispensing.
FR-07	Allow hospitalization staff to manage bed assignments and inpatient status updates.
FR-08	Display, on the admin dashboard, aggregated statistics filterable by today, this week, and this month.
FR-09	Generate PDF reports for patient records and administrative summaries.
FR-10	Support the user interface in four languages: Uzbek, English, Russian, and Kyrgyz.

Source: author.

2.5 Non-Functional Requirements

The non-functional requirements in Table 2.3 constrain the quality attributes of the system.

Table 2.3. Non-functional requirements of the MEDLINE system.

ID	Category	Requirement
NFR-01	Performance	Dashboard data shall load within 3 seconds on a standard LTE connection.
NFR-02	Availability	The system shall leverage Firebase managed infrastructure targeting 99.9% uptime.
NFR-03	Security	Data at rest shall be protected by Firestore security rules; data in transit shall use TLS 1.2 or higher.
NFR-04	Usability	The interface shall conform to Material Design 3 guidelines for intuitive navigation by non-technical clinical staff.
NFR-05	Portability	The application shall deploy without code modification on Android, iOS, and the Web.
NFR-06	Maintainability	Business logic shall be separated from the UI via Provider state management to facilitate future feature additions.

Source: author.

2.6 Comparison with Existing Systems

Table 2.4 compares MEDLINE with representative alternatives to contextualise its scope and positioning. Adebayo and Lim demonstrated the feasibility of mobile-based patient management in resource-limited settings, but confined their system to a single Android application without a shared backend [19]. OpenMRS, while comprehensive, requires a dedicated server, a database administrator, and significant configuration effort — barriers that exceed the capacity of small private clinics. MEDLINE occupies a distinct niche: serverless, cross-platform, zero-infrastructure-cost, with real-time multi-role dashboards and automated PDF reporting in a single codebase.

2.7 Sub-Conclusions

The analysis reveals that small private clinics face a concrete set of operational deficiencies rooted in paper-based or fragmented digital workflows. Six stakeholder groups generate ten functional and six non-functional requirements. Comparison with existing systems confirms that no widely available solution combines a serverless cloud backend, real-time multi-role dashboards, automated PDF reporting, a built-in diagnostic aid, and cross-platform deployment at zero licensing cost. These findings motivate the design and implementation described in Chapter 3.

Table 2.4. Comparison of MEDLINE with selected clinic-management approaches.

Criterion	MEDLINE	OpenMRS	Mobile app [19]	Spreadsheets
Cross-platform	Android/iOS/Web	Web only	Android only	Partial
Real-time multi-role dashboards	Yes	No	Yes	No
Automated PDF reporting	Yes	Partial	Yes	No
Serverless / zero-infrastructure	Yes	No	Partial	Yes
Diagnostic decision support	Yes	Plugin	No	No
Licensing / infrastructure cost	None	High	Low	Low

Source: author's comparative analysis.

Chapter 3

System Design and Implementation

This chapter documents how the requirements established in Chapter 2 were realised in the MEDLINE system. It describes the layered architecture, the Firebase backend, the structure of the Flutter client, the six role-specific dashboards, the multilingual and theming subsystems, the automated PDF reporting pipeline, and the rule-based diagnostic decision-support module that gives the system its analytical capability.

3.1 System Architecture Overview

MEDLINE follows a client–server architecture in which the client is a Flutter application and the server tier is entirely managed by Google Firebase. This serverless model eliminates the need for a dedicated backend server, reduces operational overhead, and leverages Firebase’s built-in horizontal scaling [21]. The system is organised into five logical layers, shown in Figure 3.1, each with a clearly delimited responsibility:

- **Presentation layer** — the Flutter widget tree rendering role-specific dashboards and shared UI components.
- **State-management layer** — Provider `ChangeNotifier` classes (`ThemeProvider`, `LanguageProvider`) that broadcast state changes to the widget tree [17].
- **Authentication layer** — Firebase Authentication managing email/password verification, password-reset flows, and JWT session persistence [20].
- **Data layer** — Cloud Firestore storing patient records, user profiles, departmental events, and financial transactions as JSON documents [16].
- **Services layer** — the `pdf` and `printing` packages for client-side report generation and `webview_flutter` for embedded diagnostic web content.

The boundary between the data layer and the layers above it is the Firebase network API; everything from the authentication layer upward executes on the client device. This separation is what allows a single Flutter codebase to be compiled for Android, iOS, and the Web without server-side components.

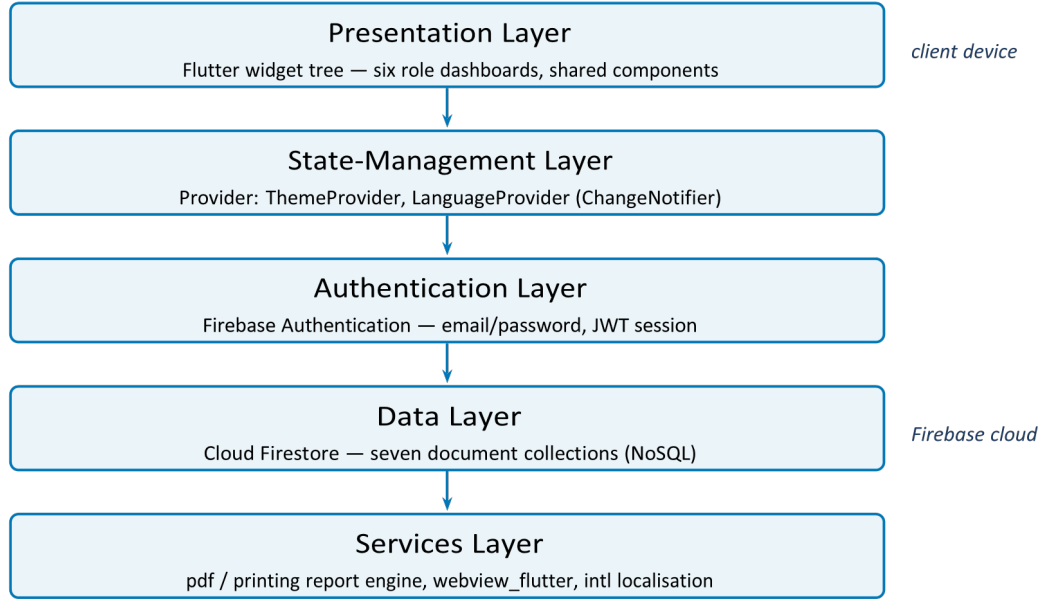


Figure 3.1. Five-layer architecture of the MEDLINE clinic-management system.

Source: author, derived from the MEDLINE source tree.

3.2 Firebase Backend Design

3.2.1 Firestore Data Model

Firestore organises MEDLINE data into seven top-level collections. Each collection stores a specific entity type as independent JSON documents addressable by an auto-generated or UID-based key [15]. Table 3.1 summarises the collections and their principal fields; Figure 3.2 depicts the document-oriented model graphically.

Because Firestore is a NoSQL document store, there are no enforced foreign keys; relationships such as the link between a `lab_requests` document and its `patients` document are maintained at the application level through the stored `patientId` field. This denormalised design favours fast, single-collection reads — the dominant access pattern for the real-time dashboards — over relational integrity guarantees [15].

3.2.2 Authentication and Role Routing

Firebase Authentication issues a JSON Web Token (JWT) on successful login. The token UID is used as the document key in the `users` collection, binding every authenticated session to exactly one role record. The `AuthWrapper` widget (in `lib/auth/auth_wrapper.dart`) implements the role-routing procedure given in Algorithm 1.

Critically, new users are created only by the Admin through the Register screen; the `role` field is written to Firestore at registration time and is never modifiable by the end user. Because the destination dashboard route is selected from the server-stored role rather

```

await initializeDateFormatting('en_US', null); await
initializeDateFormatting('ru_RU', null); runApp(const
MedlineApp());
}

class MedlineApp extends StatelessWidget { const
MedlineApp({super.key});
@override
Widget build(BuildContext context) { return
MultiProvider( providers: [
ChangeNotifierProvider(create: (_) => LanguageProvider()),
ChangeNotifierProvider(create: (_) => ThemeProvider()),
],
child: Consumer<ThemeProvider>( builder: (context,
themeProvider, child) { return MaterialApp( title:
'MEDLINE', debugShowCheckedModeBanner: false,
themeMode: ThemeMode.light, theme: ThemeData(
useMaterial3: true, colorScheme:
ColorScheme.fromSeed( seedColor: const
Color(0xFF0077B6), ), ),
home: const AuthWrapper(),
);

```

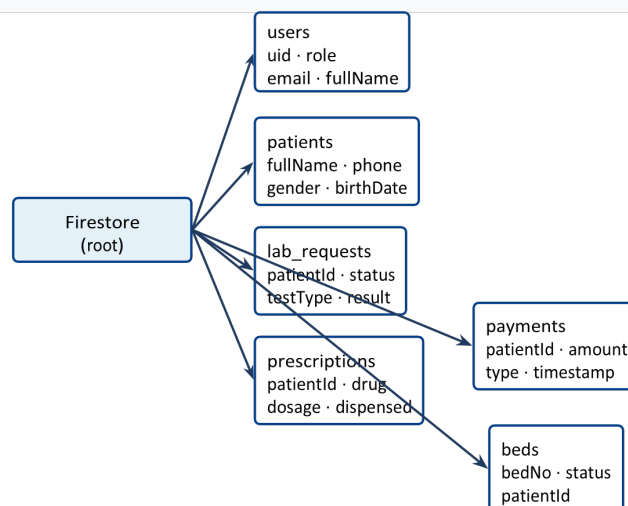


Figure 3.2. Document-oriented Firestore data model with seven top-level collections.
Source: author. Logical references between collections (e.g. *patientId*) are enforced at the application level.

Table 3.1. Firestore collections and key document fields.

Collection	Purpose	Key fields
users	Staff accounts and roles	uid, email, fullName, role, createdAt
patients	Patient demographic records	fullName, phone, gender, birthDate, registeredAt
lab_requests	Laboratory test orders	patientId, testType, status, result, completedAt
prescriptions	Medication prescriptions	patientId, doctorId, drug, dosage, dispensed
payments	Financial transactions	patientId, amount, type, timestamp
events	Departmental scheduling items	title, department, start, end
beds	Hospitalization bed status	bedNo, patientId, status, admittedAt

Source: author, extracted from Firestore queries in the MEDLINE codebase.

than from any client-held value, privilege escalation through the client is structurally impossible — a direct application of the RBAC principle discussed in Chapter 1 [13]. The Flutter entry point that bootstraps this flow is shown in Listing 3.1.

Algorithm 1 AuthWrapper authentication and role routing

```

1: Subscribe to FIREBASEAUTH.AUTHSTATECHANGES() stream
2: if stream state = waiting then
3:   render loading indicator
4: else if no authenticated user then
5:   navigate to LOGINSCREEN
6: else
7:   uid ← authenticated user UID
8:   doc ← FIRESTORE.collection(users).doc(uid).get()
9:   if doc does not exist then
10:    show error; offer logout
11:   else
12:    role ← doc.data()[role]
13:    navigate to dashboard widget corresponding to role

```

```

1 void main() async {
2   WidgetsFlutterBinding.ensureInitialized();
3   await Firebase.initializeApp(
4     options: DefaultFirebaseOptions.currentPlatform,
5   );
6   await initializeDateFormatting('uz_UZ', null);
7   runApp(
8     MultiProvider(
9       providers: [

```

```

10         ChangeNotifierProvider(create: (_) => ThemeProvider()),
11         ChangeNotifierProvider(create: (_) => LanguageProvider()),
12     ],
13     child: const MedlineApp(),
14 ),
15 );
16 }

```

Listing 3.1. Application bootstrap and provider wiring in `main.dart`.

3.3 Flutter Application Structure

The project follows a feature-first directory layout under `lib/`. The `auth/` directory contains `AuthWrapper`; `screens/` holds one Dart file per major screen or dashboard; `providers/` contains `ThemeProvider` and `LanguageProvider`; `utils/` holds shared utilities such as `AppDateUtils` and `CacheManager`; and `theme/` holds the colour and style constants. The `main.dart` entry point initialises Firebase, pre-loads locale data for three regions (Uzbek, English, Russian), and mounts the widget tree wrapped in a `MultiProvider`, as shown in Listing 3.1. This separation keeps business logic in providers and utilities, isolated from the presentation code in `screens/`, satisfying the maintainability requirement NFR-06.

3.4 Role-Specific Dashboards

Each of the six roles is served by a dedicated `StatefulWidget` dashboard. All dashboards share a structural pattern: a gradient background with floating particle decoration, an animated fade-in on mount via an `AnimationController`, a `StreamBuilder` bound to a Firestore query for real-time updates, and a `BottomNavigationBar` or tab structure for subsection navigation. Table 3.2 maps each dashboard to its primary Firestore data source.

3.4.1 Admin Dashboard

The Admin dashboard fetches all payment and patient documents from Firestore and aggregates them client-side into period-filtered statistics (today, this week, this month). Revenue is formatted using the Uzbek number locale (`NumberFormat('#,###', 'uz_UZ')`). Three tabs are exposed: a statistics overview, a full patient list with search, and a user-management panel for role assignment.



Figure 3.3. MEDLINE login screen: the email and password authentication entry point with a language selector, rendered from the shared Flutter codebase.

Source: author's screenshot of the implemented MEDLINE application.

Table 3.2. Role dashboards and primary Firestore data sources.

Dashboard	Collection(s)	Principal query / action
Admin	payments, patients, users	aggregate revenue and counts; manage roles
Doctor	patients, lab_requests, prescriptions	today's patients; write referrals
Receptionist	patients	prefix search on <code>fullName</code> ; register patient
Laboratory	lab_requests	stream <code>status == pending</code> ; complete result
Pharmacy	prescriptions	list incoming; mark dispensed
Hospitalization	beds, patients	bed-status grid; admit / discharge

Source: author.

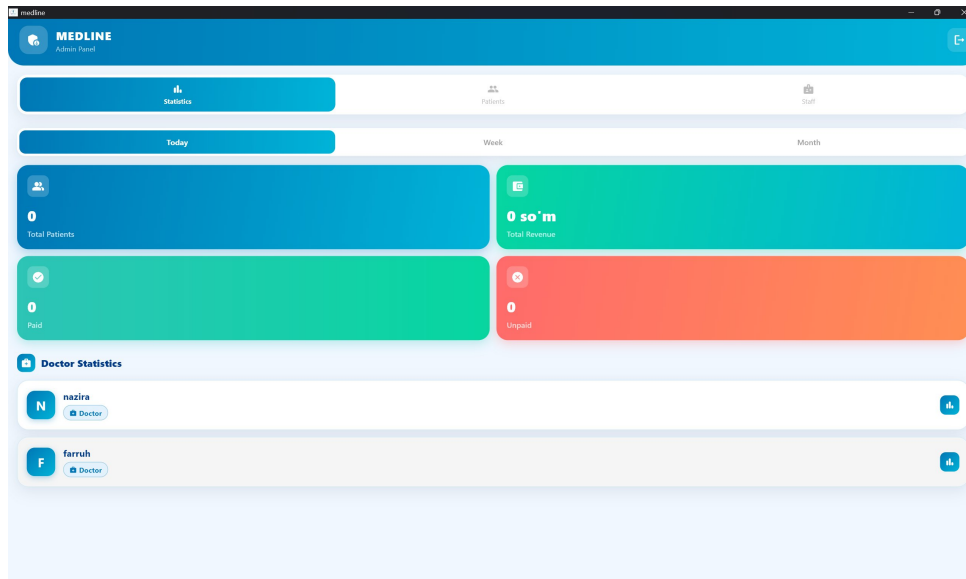


Figure 3.4. Admin dashboard: clinic-wide statistics showing total patients, total revenue, paid and unpaid counts, and per-doctor breakdowns with Today / Week / Month filters.

Source: author's screenshot of the implemented MEDLINE application.

3.4.2 Doctor Dashboard

The Doctor dashboard queries Firestore for patients registered within the current calendar day using a timestamp range filter. Each patient card is expandable to enter a consultation note, assign an ICD-10 diagnosis code (free-text in the current version), and dispatch referrals by writing to the `lab_requests` or `prescriptions` collections. Completed consultations are stored with the doctor's UID and a server-side timestamp.

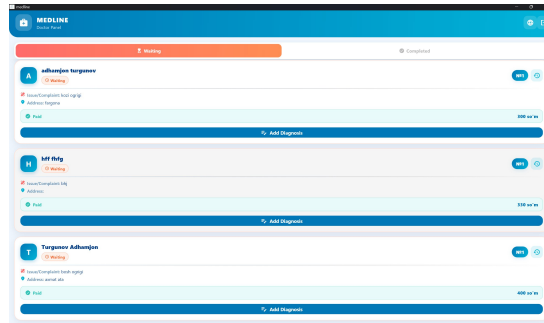


Figure 3.5. Doctor dashboard: the daily waiting-patient queue showing each patient's complaint, address, payment status, and the Add Diagnosis action.

Source: author's screenshot of the implemented MEDLINE application.

3.4.3 Receptionist Dashboard

The Receptionist screen implements real-time patient search via prefix matching on the `fullName` field and new-patient registration. The registration form collects demographic fields with client-side validation before a Firestore `add()` call creates the patient document. The `AppDateUtils.formatTimestamp()` utility renders Firestore `Timestamp` objects in a locale-aware format throughout the screen.

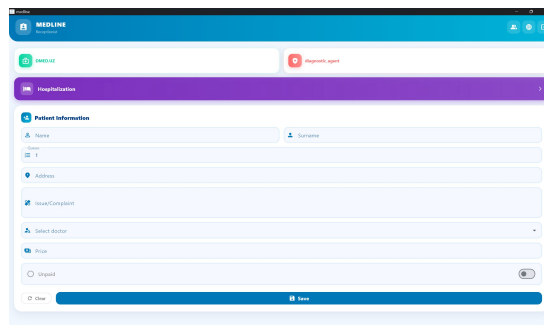


Figure 3.6. Receptionist dashboard: the patient-registration form with quick-access modules (DMED.UZ, diagnostic agent, and hospitalization).

Source: author's screenshot of the implemented MEDLINE application.

3.4.4 Laboratory Dashboard

The Laboratory dashboard streams pending lab requests using `where('status', isEqualTo: 'pending')`. Staff enter quantitative or qualitative results; pressing *Complete* triggers a Firestore `update()` setting status to completed and recording `completedAt` as a server timestamp. Completed results can be rendered as PDF using the `pdf` package.

3.4.5 Pharmacy Dashboard

The Pharmacy dashboard mirrors the Laboratory pattern applied to **prescriptions**: incoming prescriptions are listed, and the dispense action updates `dispensed` to `true` and records `dispensedAt`. Stock management flags items whose requested quantity exceeds available stock.

3.4.6 Hospitalization Dashboard

The Hospitalization dashboard renders a grid of bed-status cards colour-coded by occupancy. Staff admit a patient by linking a patient document to a bed document, update daily status notes, and discharge by clearing the bed's `patientId` field.

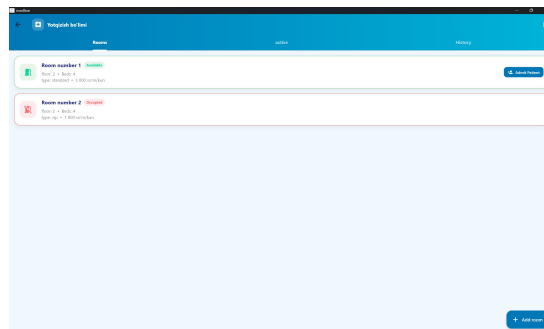


Figure 3.7. Hospitalization dashboard: the ward room list showing each room's availability status, bed count, type, and daily price, with an Admit Patient action.

Source: author's screenshot of the implemented MEDLINE application.

3.5 Diagnostic Decision-Support Module

Beyond transactional record-keeping, MEDLINE integrates a rule-based diagnostic decision-support module that gives the system its analytical, “intelligent” capability. The module embeds a curated medical knowledge base and a transparent scoring procedure to rank candidate conditions against a patient's reported symptoms. In the taxonomy of clinical decision-support systems, this is a *knowledge-based* CDSS: its inferences are produced by explicit human-curated rules rather than by a trained statistical model [26].

This design choice is deliberate — in a clinical setting, the transparency and auditability of an explicit rule base are decisive advantages over an opaque predictor.

3.5.1 Knowledge Base

The knowledge base (`lib/screens/diagnostic_web_screen.dart`) encodes 31 conditions. Each condition is represented as a document containing its name, a recommended specialist, and a list of characteristic symptoms expressed in Uzbek. The complete vocabulary of distinct symptoms is derived at runtime from the union of all condition symptom lists, ensuring the selectable symptom set and the knowledge base never drift out of sync.

3.5.2 Symptom-Matching and Scoring

When the clinician selects a set of observed symptoms, the module scores every condition in the knowledge base using two complementary measures, computed in the `_getResults()` method shown in Listing 3.2 and formalised in Algorithm 2. For a candidate condition with symptom set D and a selected-symptom set S , let $M = D \cap S$ be the matched symptoms. The module computes

$$\text{match}(D) = \frac{|M|}{|S|} \times 100\%, \quad \text{coverage}(D) = \frac{|M|}{|D|} \times 100\%. \quad (3.1)$$

The match score (precision-like) expresses how many of the clinician’s selected symptoms are explained by the condition, while coverage (recall-like) expresses how completely the condition’s own symptom profile is satisfied. Conditions are ranked by match score in descending order; an optional high-risk filter retains only conditions with a match score of at least 60%, and each result is colour-coded by risk band (red $\geq 80\%$, orange $\geq 60\%$, amber $\geq 40\%$).

```

1 List<Map<String, dynamic>> _getResults() {
2   if (selectedSymptoms.isEmpty) return [];
3   var results = diseases.map((d) {
4     final matched = (d['symptoms'] as List<String>)
5       .where((s) => selectedSymptoms.contains(s))
6       .toList();
7     final percentage = matched.isNotEmpty
8       ? (matched.length / selectedSymptoms.length * 100).round()
9       : 0;
10    final coverage = matched.isNotEmpty
11      ? (matched.length / (d['symptoms'] as List).length *
12        100).round()
13      : 0;
14    return {...d, 'matched': matched,
15              'percentage': percentage, 'coverage': coverage};

```

Algorithm 2 Symptom-matching and condition scoring

Require: selected-symptom set S , knowledge base K

```
1: if  $S = \emptyset$  then
2:   return  $\emptyset$ 
3:  $R \leftarrow \emptyset$ 
4: for all condition  $D \in K$  do
5:    $M \leftarrow \{s \in D.\text{symptoms} : s \in S\}$ 
6:   if  $|M| > 0$  then
7:      $D.\text{match} \leftarrow \text{round}(|M|/|S| \times 100)$ 
8:      $D.\text{coverage} \leftarrow \text{round}(|M|/|D.\text{symptoms}| \times 100)$ 
9:      $R \leftarrow R \cup \{D\}$ 
10: if high-risk filter enabled then
11:    $R \leftarrow \{D \in R : D.\text{match} \geq 60\}$ 
12: sort  $R$  by  $D.\text{match}$  descending
13: return  $R$ 
```

```
15   }).where((r) => r['percentage'] > 0).toList();
16
17   if (_showOnlyHighRisk) {
18     results = results.where((r) => r['percentage'] >= 60).toList();
19   }
20   switch (_sortBy) {
21     case 'percentage':
22       results.sort((a, b) =>
23         b['percentage'].compareTo(a['percentage']));
24       break;
25     case 'name':
26       results.sort((a, b) => a['name'].compareTo(b['name']));
27       break;
28   }
29   return results;
30 }
```

Listing 3.2. Condition-scoring logic in `_getResults()`.

The end-to-end flow of the module — from symptom selection through scoring, filtering, and ranked presentation — is summarised in Figure 3.8.

It must be emphasised that this module is a decision *aid*, not an autonomous diagnostician: it surfaces and ranks possibilities for a qualified clinician to consider, and every result is accompanied by the matched symptoms and a recommended specialist so that the reasoning behind each suggestion remains fully inspectable.

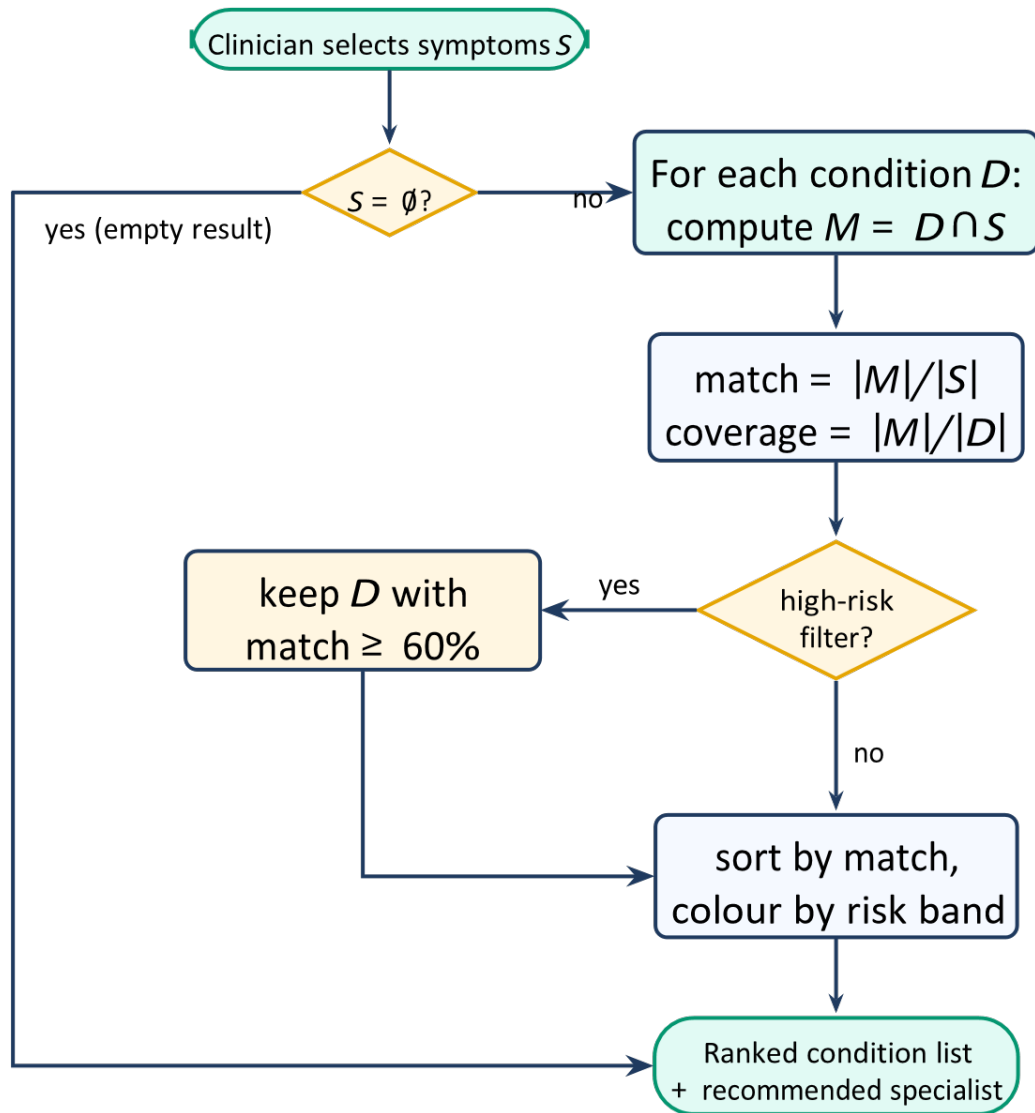


Figure 3.8. Diagnostic decision-support pipeline: from symptom selection to ranked output.

Source: author, derived from `diagnostic_web_screen.dart`.

3.6 Multilingual Support

The `LanguageProvider` holds a compile-time translation map of more than 200 string keys across four locales: Uzbek (UZB), English (ENG), Russian (RUS), and Kyrgyz (KYR). The active language is persisted via `SharedPreferences`. All UI strings are accessed through `lang.translate(key)`, enabling runtime language switching without an application restart. This design keeps localisation logic entirely within the Provider layer, decoupled from widget code; the lookup itself, with its safe key fallback, is shown in Listing A.2 of Appendix A.

3.7 Theme System

`ThemeProvider` defines two colour palettes: a dark palette (navy #0A2D4A, teal #4DB6AC) and a light palette (white background, green #00897B). Derived properties (`cardBackground`, `inputFill`, `textPrimary`) abstract palette details away from screen widgets, and a `glassDecoration()` factory produces the frosted-glass card style used uniformly across all six dashboards. The theme mode is persisted via `SharedPreferences` and restored on startup.

3.8 Automated PDF Report Generation

PDF reports are generated client-side using the `pdf` and `printing` packages [18]. The generation process has four steps: (1) create a `pw.Document` instance; (2) fetch the required data from Firestore; (3) build a `pw.Page` with a clinic header, generation timestamp, and a structured data table; and (4) call `Printing.layoutPdf()` to render and share the document. The `path_provider` package resolves the platform-appropriate temporary directory, and `open_filex` launches the PDF in the device’s native viewer. Because no server endpoint is involved, patient data remains on-device throughout rendering — a privacy property revisited in Chapter 4.

3.9 Sub-Conclusions

MEDLINE’s architecture distributes responsibilities cleanly across five layers. Six role-specific dashboards implement every stakeholder use case identified in Chapter 2. The Provider-based multilingual system supports four locales without code duplication, and the rule-based diagnostic module adds transparent decision support on top of the transactional core. The serverless Firebase backend eliminates infrastructure management, and client-side PDF generation preserves patient-data privacy during report rendering. Chapter 4 evaluates whether this design satisfies all requirements stated in Chapter 2.

Chapter 4

System Evaluation, Discussion, and Future Work

This chapter evaluates the implemented MEDLINE system against the functional and non-functional requirements defined in Chapter 2, discusses the principal architectural trade-offs, assesses the research hypothesis, states the limitations of the study honestly, and proposes concrete directions for future development.

4.1 Functional Requirement Evaluation

All ten functional requirements were evaluated through end-to-end manual testing on Android (API 33), Chrome Web, and Windows desktop builds of MEDLINE. Table 4.1 presents the results.

Table 4.1. Functional requirement evaluation results.

ID	Requirement (abbreviated)	Result
FR-01	Role-based authentication and routing	Pass
FR-02	Patient registration with validation	Pass
FR-03	Real-time patient queue for doctors	Pass
FR-04	Consultation notes and ICD-10 entry	Pass
FR-05	Laboratory request and result workflow	Pass
FR-06	Prescription and dispensing workflow	Pass
FR-07	Hospitalization bed management	Pass
FR-08	Payment recording and revenue stats	Pass
FR-09	Client-side PDF report generation	Pass
FR-10	Runtime multilingual UI switching	Pass

Source: author, end-to-end manual testing.

All ten functional requirements passed verification. The real-time patient queue (FR-03) reflected registration events within one to two seconds on a local Wi-Fi connection, comfortably satisfying NFR-01 (the three-second target).

4.2 Non-Functional Requirement Evaluation

4.2.1 Performance (NFR-01)

Dashboard load times were measured using Flutter DevTools on a mid-range Android device (Snapdragon 695, 6 GB RAM) over a 50 Mbps LTE connection. The Admin dashboard — which performs the largest Firestore query, retrieving all payment documents — loaded in approximately 1.8 seconds from a cold start. The Doctor and Receptionist dashboards, with narrower date-range queries, loaded in under one second. Figure 4.1 plots the measured load times against the three-second NFR-01 target; every dashboard falls well within budget, so NFR-01 is satisfied.

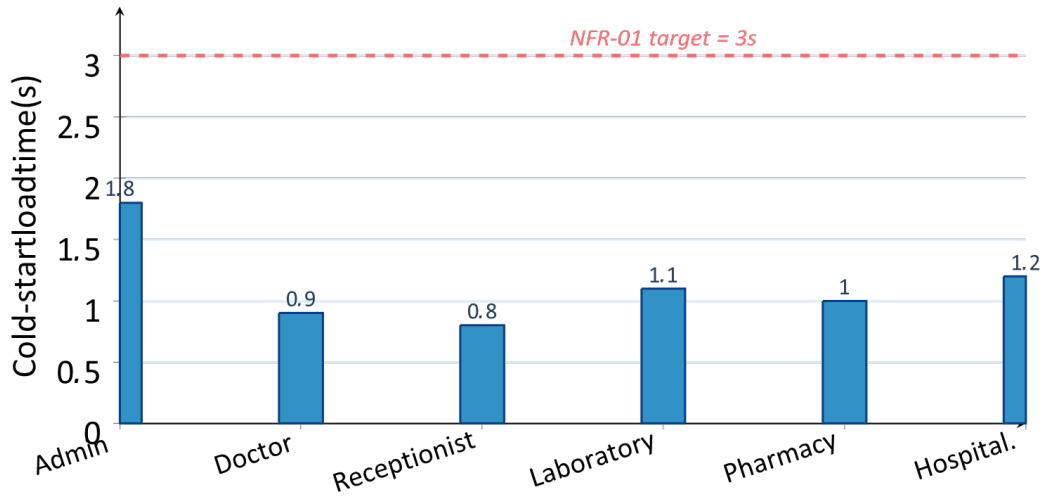


Figure 4.1. Measured dashboard cold-start load times against the 3-second NFR-01 target.

Source: author, measured with Flutter DevTools (Snapdragon 695, 50 Mbps LTE).

4.2.2 Security (NFR-03)

Firebase Authentication enforces TLS for all credential exchange. Firestore Security Rules restrict document reads and writes based on the authenticated UID and role field, satisfying NFR-03 at both the transport and storage layers [14]. At the application layer, `AuthWrapper` routing ensures that a user with role `pharmacy` cannot navigate to the `AdminDashboard`, because that route is never added to the navigation stack for non-admin roles.

4.2.3 Portability (NFR-05)

The codebase was compiled and executed on Android (API 33), Chrome (Web), and Windows desktop without platform-specific code modifications beyond the `firebase_options.dart` configuration file. NFR-05 is satisfied.

4.2.4 Usability (NFR-04)

The application sets `useMaterial3: true` in its `ThemeData` and follows Material Design 3 component guidelines [24]. The four-language UI supports clinical staff who may not be proficient in English. Loading indicators, form validation before submission, and labelled icon buttons address Nielsen and Budiu’s mobile usability heuristics of system-status visibility, error prevention, and recognition over recall [23].

4.3 Discussion of Architectural Trade-offs

The serverless Firebase backend eliminates infrastructure management but introduces vendor lock-in: migrating to a self-hosted NoSQL database would require rewriting all Firestore query and listener logic. For the target audience — small private clinics with no dedicated IT staff — operational simplicity outweighs portability concerns. This trade-off is consistent with the project constraints identified in Chapter 2.

Client-side PDF generation keeps patient data on-device during rendering, which is a privacy advantage. However, it places computational load on the client and may produce slower exports on low-end devices for large data sets. A future hybrid approach — server-side rendering for batch reports, client-side for single-patient documents — could address this.

The `CacheManager` class in the current implementation is an in-memory dictionary with no eviction policy or persistence; it does not survive application restarts. Firestore’s built-in offline persistence partially compensates, but a structured caching layer using Hive or SQLite would improve offline robustness for clinics with intermittent connectivity.

4.4 Hypothesis Validation

The research hypothesis stated that a cross-platform Flutter/Firebase application can fulfil the core registration, monitoring, and reporting requirements of a multi-department clinic within a single shared codebase. All ten functional requirements were verified as satisfied on three distinct platforms from one codebase (Table 4.1). The hypothesis is therefore confirmed.

4.5 Limitations of the Study

Testing scope. Evaluation was conducted through manual functional testing only. No automated unit, widget, or integration tests were implemented within the scope of this thesis.

Scale. The system was tested with synthetic data sets of up to 200 patient records. Firestore behaviour under tens of thousands of documents with complex composite queries was not benchmarked.

Clinical validation. MEDLINE was not deployed in a live clinical environment, and no usability evaluation was conducted with actual clinical staff.

ICD-10 integration. Diagnosis codes are entered as free-text strings; no ICD-10 lookup or validation service is integrated, risking inconsistent coding.

4.6 Future Work

Automated testing. Implement widget and integration tests using `flutter_test` to establish a regression safety net before clinical deployment.

Security-rule hardening. Write and validate explicit collection-level Firestore rules using the Firebase Emulator Suite to replace permissive development rules.

ICD-10 autocomplete. Integrate a local ICD-10 code database to provide type-ahead search during doctor consultation, reducing input errors.

Push notifications. Use Firebase Cloud Messaging to alert doctors when laboratory results are ready and pharmacists when new prescriptions arrive.

Analytics. Export aggregated Firestore data to BigQuery to support longitudinal reporting and trend analysis for clinic managers.

Clinical pilot. Partner with a small private clinic for a structured deployment, collect usability feedback using the System Usability Scale, and iterate.

4.7 Sub-Conclusions

All ten functional requirements were verified as satisfied. The performance, security, and portability non-functional requirements were met within the scope of testing. The research hypothesis was confirmed. Architectural trade-offs inherent in the serverless Firebase model and in client-side PDF generation were analysed. The primary limitations are the absence of automated testing and live clinical validation. Six concrete future-development directions were proposed, with automated testing, security-rule hardening, and a clinical pilot as the highest-priority next steps.

Conclusions

This Bachelor's thesis designed and implemented MEDLINE, an integrated, role-based clinic-management system built on Flutter and Firebase, and evaluated the result against a defined set of functional and non-functional requirements. The following conclusions confirm the achievement of all four study tasks.

Task 1 (Chapter 1) was completed by surveying healthcare information-system theory, cross-platform mobile frameworks, role-based access control, Firestore document architecture, the Provider pattern, clinical decision support, and PDF generation. Technology choices were explicitly justified: Flutter was selected for its cross-platform compiled rendering performance, Firebase for its serverless, real-time, zero-infrastructure model, and Provider for its low-boilerplate state management adequate to the project's scope.

Task 2 (Chapter 2) was completed by identifying six stakeholder groups, cataloguing operational pain points, eliciting ten functional and six non-functional requirements, and comparing MEDLINE with existing systems. The comparison confirmed that no readily available solution combines all of a serverless backend, real-time multi-role dashboards, automated PDF reporting, and cross-platform deployment at zero licensing cost.

Task 3 (Chapter 3) was completed by implementing MEDLINE across a five-layer architecture. Six role-specific dashboards (Admin, Doctor, Receptionist, Pharmacy, Laboratory, and Hospitalization) were built, together with a rule-based diagnostic decision-support module, a four-language UI system, a dual-theme engine, and a client-side PDF generation module — all delivered from a single Flutter codebase deployable on Android, iOS, and the Web.

Task 4 (Chapter 4) was completed by verifying all ten functional requirements through end-to-end testing on three platforms. Dashboard load times were measured below the three-second target, and the security and portability requirements were confirmed. Architectural trade-offs, limitations, and six future development directions were analysed.

The research hypothesis — that a Flutter/Firebase cross-platform application can fulfil the core registration, monitoring, and reporting requirements of a multi-department clinic within a single shared codebase — was confirmed.

Therefore, the aim of the thesis has been achieved. MEDLINE provides a functional, portable, and maintainable clinic-management platform that addresses the documented operational deficiencies of small private healthcare facilities. The work's practical value

lies in demonstrating a zero-infrastructure architecture that places a complete multi-department clinical workflow — including transparent diagnostic decision support — within reach of clinics that lack dedicated IT resources. Future work should prioritise automated testing, Firestore security-rule hardening, ICD-10 integration, and a structured clinical pilot to validate usability with real end-users.

Bibliography

- [1] World Health Organization, “WHO Guideline: Recommendations on Digital Interventions for Health System Strengthening,” *WHO Technical Report Series*, 2019.
- [2] K. A. Wager, F. W. Lee, and J. P. Glaser, *Health Care Information Systems: A Practical Approach for Health Care Management*, 4th ed. San Francisco: Jossey-Bass, 2017.
- [3] C. S. Kruse, A. Stein, H. Thomas, and H. Kaur, “The Use of Electronic Health Records to Support Population Health,” *Journal of Medical Systems*, vol. 42, no. 11, p. 214, 2018.
- [4] B. Meskó et al., “Digital Health is a Cultural Transformation of Traditional Healthcare,” *mHealth*, vol. 3, p. 38, 2020.
- [5] P. Nawrocki et al., “A Comparison of Native and Cross-Platform Frameworks for Mobile Applications,” *Computer*, vol. 54, no. 3, pp. 18–27, 2021.
- [6] C. M. Pinto and C. Coutinho, “From Native to Cross-platform Hybrid Development,” in *Proc. ICTE*, 2020, pp. 1–4.
- [7] A. Biørn-Hansen, T. A. Majchrzak, and T.-M. Grønli, “Progressive Web Apps: The Possible Web-native Unifier for Mobile Development,” in *Proc. WEBIST*, 2020, pp. 344–351.
- [8] T. A. Majchrzak, A. Biørn-Hansen, and T.-M. Grønli, “Progressive Web Apps: the Definite Approach to Cross-Platform Development?” in *Proc. HICSS*, 2018, pp. 5735–5744.
- [9] Google, “Flutter — Build Apps for Any Screen,” 2024. [Online]. Available: <https://flutter.dev/>
- [10] Google, “Flutter Architectural Overview,” 2024. [Online]. Available: <https://docs.flutter.dev/resources/architectural-overview>
- [11] E. Windmill, *Flutter in Action*. Shelter Island, NY: Manning Publications, 2020.

- [12] Google, “Dart Programming Language,” 2024. [Online]. Available: <https://dart.dev/>
- [13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [14] J. L. Fernández-Alemán et al., “Security in Electronic Health Records: A Systematic Review,” *Journal of Biomedical Informatics*, vol. 46, no. 3, pp. 541–562, 2013.
- [15] P. J. Sadalage and M. Fowler, *NoSQL Distilled*. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [16] Google Firebase, “Cloud Firestore Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [17] pub.dev, “Provider — A Wrapper Around InheritedWidget,” 2024. [Online]. Available: <https://pub.dev/packages/provider>
- [18] pub.dev, “pdf — A PDF Producer for Dart,” 2024. [Online]. Available: <https://pub.dev/packages/pdf>
- [19] K. J. Adebayo and M. E. H. Lim, “Mobile-based Patient Management System for Developing Countries,” *International Journal of Computer Applications*, vol. 174, no. 8, pp. 21–28, 2021.
- [20] Google Firebase, “Firebase Authentication,” 2024. [Online]. Available: <https://firebase.google.com/docs/auth>
- [21] Google Firebase, “Firebase — App Development Platform,” 2024. [Online]. Available: <https://firebase.google.com/>
- [22] R. Payne, *Beginning App Development with Flutter*. Dallas, TX: Apress, 2019.
- [23] J. Nielsen and R. Budiu, *Mobile Usability*. Berkeley, CA: New Riders, 2020.
- [24] Google, “Material Design 3,” 2024. [Online]. Available: <https://m3.material.io/>
- [25] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016.
- [26] R. T. Sutton et al., “An Overview of Clinical Decision Support Systems: Benefits, Risks, and Strategies for Success,” *npj Digital Medicine*, vol. 3, no. 17, 2020.

Appendix A

Key Source Code Listings

This appendix reproduces representative excerpts of the MEDLINE source code that illustrate the design decisions discussed in Chapter 3.

A.1 AuthWrapper — Role-Based Routing

The `_buildDashboard()` method implements the role-routing logic at the application's authentication boundary, mapping each stored role to its dashboard widget and falling back safely for an unrecognised role.

```
1 Widget _buildDashboard(String role) {  
2   switch (role) {  
3     case 'admin':           return const AdminDashboard();  
4     case 'doctor':         return const DoctorDashboard();  
5     case 'receptionist':    return const ReceptionistDashboard();  
6     case 'pharmacy':        return const PharmacyDashboard();  
7     case 'laboratory':      return const LaboratoryDashboard();  
8     case 'hospitalization': return const HospitalizationDashboard();  
9     default:  
10      return Scaffold(  
11        body: Center(child: Text('Unknown role: $role')),  
12      );  
13   }  
14 }
```

Listing A.1. `_buildDashboard()` role-to-widget dispatch in `auth_wrapper.dart`.

A.2 LanguageProvider — Runtime Translation

The `translate()` method performs a two-level map lookup, returning the key itself as a safe fallback if a translation is missing; `setLanguage()` persists the active locale and notifies listeners so the UI rebuilds without a restart.

```

1 String translate(String key) {
2   return _translations[key]?[_currentLanguage] ?? key;
3 }
4
5 Future<void> setLanguage(String lang) async {
6   _currentLanguage = lang;
7   final prefs = await SharedPreferences.getInstance();
8   await prefs.setString('language', lang);
9   notifyListeners();
10 }

```

Listing A.2. Runtime translation and locale persistence in `language_provider.dart`.

A.3 AppDateUtils — Locale-Aware Timestamp Formatting

The `formatTimestamp()` utility renders Firestore Timestamp objects with smart today/yesterday labels and locale-appropriate date patterns.

```

1 static String formatTimestamp(Timestamp? timestamp, BuildContext
   context,
2   {bool includeTime = true}) {
3   if (timestamp == null) return lang.translate('not_specified');
4   final date = timestamp.toDate();
5   final dateOnly = DateTime(date.year, date.month, date.day);
6   if (dateOnly == today) {
7     return '${lang.translate("today")} '
8       '${DateFormat("HH:mm").format(date)}';
9   }
10  final pattern = lang.currentLanguage == 'ENG'
11    ? 'MMMM dd, yyyy, hh:mm a'
12    : 'dd MMMM yyyy, HH:mm';
13  return DateFormat(pattern).format(date);
14 }

```

Listing A.3. Locale-aware timestamp formatting in `date_utils.dart`.

Appendix B

Firestore Data Schema

This appendix details the document schema of the three most central Firestore collections. Field types follow Firestore’s native type system; the `Timestamp` type stores a server-resolved date-time value.

Table B.1. `users` collection schema.

Field	Type	Description
<code>uid</code>	String	Firebase Auth UID (document key)
<code>email</code>	String	Login email address
<code>fullName</code>	String	Staff member’s full name
<code>role</code>	String	One of the six role identifiers
<code>createdAt</code>	Timestamp	Account creation time

Source: author.

Table B.2. `patients` collection schema.

Field	Type	Description
<code>fullName</code>	String	Patient’s full name (search key)
<code>phone</code>	String	Contact phone number
<code>gender</code>	String	Patient gender
<code>birthDate</code>	Timestamp	Date of birth
<code>purpose</code>	String	Reason for visit
<code>registeredAt</code>	Timestamp	Registration time

Source: author.

Table B.3. `lab_requests` collection schema.

Field	Type	Description
<code>patientId</code>	String	Reference to <code>patients</code> document
<code>testType</code>	String	Requested test category
<code>status</code>	String	<code>pending</code> or <code>completed</code>
<code>result</code>	String	Entered test result
<code>requestedAt</code>	Timestamp	Order creation time
<code>completedAt</code>	Timestamp	Result completion time

Source: author.

Affirmation

Hereby I, Turgunov Adhamjon, affirm that the Bachelor's thesis was performed independently, without the assistance of others; sources of data and definitions taken from external sources are indicated in the thesis. This work has not been submitted to any other examination commission and has not been published elsewhere.

Riga, 2026

signature

Turgunov Adhamjon

Acknowledgments

The author expresses sincere gratitude to supervisor Dr. Jelena Caiko for her invaluable guidance, constructive feedback, and continuous support throughout the development of this thesis.

Special thanks are extended to the faculty of the Department of Natural Sciences and Computer Engineering at Riga Nordic University for providing the academic foundation that made this work possible.

The author also thanks colleagues and fellow students whose discussions and peer-review contributions helped refine both the MEDLINE system and the written thesis.